Status	Finished
Started	Saturday, 5 September 2026, 1:48 PM
Completed	Saturday, 5 September 2026, 2:47 PM
Duration	59 mins 54 secs
Marks	60.00/60.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1

Correct

Mark 5.00 out of 5.00

For a **4-bit signed 2's complement system**, what is the result of adding 7 (0111) and 1 (0001)? It can be explained as:

7 (0111) + 1 (0001) = 1000 (interpreted as [-8]). This is [an overflow] because the valid range is [-8] to [7] . (It is a [positive] overflow.)

[8] [-7] [-15] [7] [-8] [0] [15]

[no overflow]

[negative] [none]

Your answer is correct.

The correct answer is:
For a **4-bit signed 2's complement system**, what is the result of adding 7 (0111) and 1 (0001)? It can be explained as:

7 (0111) + 1 (0001) = 1000 (interpreted as [-8]). This is [an overflow] because the valid range is [-8] to [7]. (It is a [positive] overflow.)

Question 2

Correct

Mark 5.00 out of 5.00

For a **4-bit unsigned 2's complement system**, what is the result of adding 7 (0111) and 1 (0001)? It can be explained as:

7 (0111) + 1 (0001) = 1000 (interpreted as [8]). This is [not overflow] because the valid range is [0] to [15] . (It is a [none] overflow.)

[7] [-8] [-15] [15] [-7] [0] [8]

[an overflow]

[negative] [positive]

Your answer is correct.

The correct answer is:
For a **4-bit unsigned 2's complement system**, what is the result of adding 7 (0111) and 1 (0001)? It can be explained as:

7 (0111) + 1 (0001) = 1000 (interpreted as [8]). This is [not overflow] because the valid range is [0] to [15]. (It is a [none] overflow.)

Question 3

Correct

Mark 20.00 out of 20.00

In an **8-bit** computer, the machine executes
127 + 1

Unsigned Check: `127 + 1 = 128`, which [is] in between [0] and [255], so it is [valid].

Carry is → [no].

Binary: [01111111] (+) [00000001] = [10000000].

Then [carry] = [0].

Signed Check: `127 + 1 = 128`, which is [greater] than [127], so it is [invalid].

Binary: [01111111] (+) [00000001] = [10000000].

Then [overflow] = [1]. It is [Positive Overflow]

Question 4

Correct

Mark 20.00 out of 20.00

In an **8-bit** computer, the machine executes
255 + 1

Unsigned Check: `255 + 1 = 256`, which [is not] in between [0] and [255], so it is [invalid].

Carry is → [yes].

Binary: [11111111] (+) [00000001] = [00000000].

Then [carry] = [1].

Signed Check: `255 + 1 = 256`, Interpret `0xFF` (255) as signed [-1],

its binary is: [11111111] **(8-bit two's complement arithmetic of 1)**.

Binary: [11111111] (+) [00000001] = [00000000], so it is [valid].

Then [overflow] = [0].

Question 5

Correct

Mark 10.00 out of 10.00

In an **8-bit** computer, the machine executes
-128 - 1

Unsigned Check: `−128 − 1 = 128 + 255?`

`−128` unsigned represents **128.**
The CPU stores only the binary pattern `1000 0000`. That exact same pattern *simultaneously* means two different numbers, depending entirely on how the programmer chooses to interpret it. When the CPU is told to do unsigned math, it reads the 8 bits as normal binary place values:

```
  Bit position:   7   6   5   4   3   2   1   0
                ┌───┬───┬───┬───┬───┬───┬───┬───┐
      0x80      | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | = 128
      0xFF      | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | = 255
                └───┴───┴───┴───┴───┴───┴───┴───┘
```

Let's look at how negative numbers count **downward** in 8-bit signed binary:

Signed Value	Bit Pattern	Unsigned Value of that Pattern
-1	`1111 1111`	255
-2	`1111 1110`	254
-3	`1111 1101`	253
...	...	...
-128	`1000 0000`	128
-129	*Cannot exist in 8 bits*	(Wraps around)

`−128` unsigned represents **128.**
`−1` unsigned represents **255 (8-bit two's complement arithmetic of 1).**

Notice the pattern: As the **signed number** gets more negative, the **unsigned** interpretation of the *same bits* gets smaller. It stops at `−128` (`1000 0000`) because that is exactly the halfway point of the 256 available patterns.

Binary of - 128 - 1: 10000000 ✓ (+) 11111111 ✓ = 01111111 ✓ .

Then carry ✓ = 1 ✓ .

Signed Check: `−128 − 1 = −128 + (−1)?`

Binary: 10000000 ✓ (+) 11111111 ✓ = 01111111 ✓ .

Then overflow ✓ = 1 ✓ . It is negative ✓ overflow.